

3 Preliminary Notation

Datasets. Let $\mathcal{X}$ be the set of all possible *prompts*, and let $\mathcal{Y}$ be the set of all possible *responses*, which we also call *labels*. An element of $\mathcal{X} \times \mathcal{Y}$ is a *labeled prompt*. A subset of $\mathcal{X}$ is a *prompt set*, and a subset of $\mathcal{X} \times \mathcal{Y}$ is a *dataset*.

For any prompt set P , let $P(x)$ denote the number of times prompt x appears in P , and for any dataset D , let $D(x, y)$ denote the number of times labeled prompt (x, y) appears in D . Typically we have $P(x) \in \{0, 1\}$ and $D(x, y) \in \{0, 1\}$. However, we also allow datasets to contain multiple copies of the same element, where the multiplicity, or *weight*, of an element can be any non-negative real number, i.e., $D(x, y) \in \mathbb{R}_+$. We write $(x, y) \in D$ if and only if $D(x, y) > 0$ and $|D| = \sum_{x, y} D(x, y)$. Datasets with general weights are formed by using the weighted union operation: If D_0 and D_1 are datasets, and $\lambda_0, \lambda_1 > 0$, then $D = \lambda_0 D_0 \uplus \lambda_1 D_1$ is the dataset defined by $D(x, y) = \lambda_0 D_0(x, y) + \lambda_1 D_1(x, y)$.

For any dataset D let $D(y|x) = D(x, y) / \sum_{y'} D(x, y')$ be the fraction of labeled prompts in D with prompt x that have response y . Define $D(y|x) = 0$ if $\sum_{y'} D(x, y') = 0$.

LLMs. A large language model, or *LLM*, is a function that maps each prompt in $\mathcal{X}$ to a distribution on the responses $\mathcal{Y}$. We will denote LLMs by the symbol g , and let $g(x)$ denote the distribution over labels $\mathcal{Y}$ of g when evaluated on prompt x .

Let $q : \mathcal{X} \times \mathcal{Y} \rightarrow \{0, 1\}$ be the *quality* function, where $q(x, y) = 1$ indicates that y is a good response to prompt x .

4 Problem Setting

We consider a setting where a sequence of LLMs $g_1, g_2, \dots$ are learned on a sequence of datasets $\mathcal{D}_1, \mathcal{D}_2, \dots$. Given a prompt set P , our high-level goal is to produce an LLM that generates high quality responses for every prompt in P . We illustrate this meta-algorithm in Setting 1.

Unlike classical learning, where the learner has access to samples from the target distribution, we assume that the learner only has access to labeled examples constructed by a data generation procedure that we control, denoted by the function **GenerateData**. Data generation might make use of synthetic data, produced by the previous generation’s LLM g_{t-1} , and exogenous (i.e., non-synthetic) signals.

In order to formalize our goal, we make precise the capabilities of **learner**, the capabilities of **GenerateData**, and our notion of quality.

Setting 1 Data Generation Problem

Given: Prompt set P , number of iterations T .

- 1: $g_0 = \perp$
 - 2: **for** $t = 1, 2, \dots, T$ **do**
 - 3: $D = \text{GenerateData}(P, g_{t-1})$
 - 4: $\mathcal{D}_t = \mathcal{D}_{t-1} \uplus D$
 - 5: $g_t = \text{learner}(\mathcal{D}_t)$
 - 6: **end for**
 - 7: Output g_T .
-

4.1 Strong Learning

We first introduce the concept of a strong learner.

Definition 1 (Strong Learner). For any LLM g let $g(y|x)$ be the probability that the distribution $g(x)$ assigns to response y . The function **learner** takes as input a dataset $D \subset \mathcal{X} \times \mathcal{Y}$ and outputs an LLM g such that $g(y|x) = D(y|x)$ for all $(x, y) \in D$.

The procedure **learner** trains an LLM that matches the conditional probability of each response given a prompt in the input dataset. That is, we assume that the model class has the capacity to match this distribution exactly, and the learning procedure can find the model parameters that perfectly fit the data. This assumption is motivated by the fact that deep neural networks instantiate all modern LLMs and are both theoretically capable of approximating arbitrary functions [Maierov and Pinkus, 1999] and frequently observed to fit their training inputs [Zhang et al., 2021].

While LLMs are powerful, the largest models contain billions of parameters and are extremely expensive to train. Thus, training T state-of-the-art models from scratch is prohibitively expensive. In contrast, given model g_{t-1} trained on $\mathcal{D}_{t-1}$, it is significantly less expensive to train a model g_t on $\mathcal{D}_{t-1} \uplus D$, for some choice D . In other words, our setting models continued training, where the training mixture for the next LLM is constructed by augmenting the existing data mixture with new examples.

4.2 Data Generation

Creating data for the next generation of an LLM might involve making use of synthetic data produced by the current generation of the LLM. To avoid model collapse, some degree of data curation happens in practice. This curation may make use of an exogenous signal previously unknown to our training algorithm. Curation may also take the form of evaluating the quality of existing synthetic data. We discuss each of these capabilities in greater detail.

Synthetic Data. Given an LLM g , and a prompt x , we can generate a synthetic response for x by sampling from distribution $g(x)$. Overall, we assume that synthetic data generation is relatively inexpensive, and permit data generation procedures that make calls to previously-trained LLMs.

Noisy Filter. We assume that we can noisily partition a dataset into labeled and unlabeled prompts based on the quality function.

Definition 2 (γ -noisy Filter). Let $\gamma \in [0, 1]$. The function $\text{filter}_\gamma(D)$ takes as input a dataset $D \subseteq \mathcal{X} \times \mathcal{Y}$ and outputs $(S^+, P^-) = \text{filter}_\gamma(D)$, where the following holds with independent probability for each $(x, y) \in D$:

- If $q(x, y) = 1$ then $(x, y) \in S^+$ with probability at least γ , and otherwise $x \in P^-$.
- If $q(x, y) = 0$ then $x \in P^-$.

In other words, γ is a lower bound on the recall of filter_γ for recognizing high-quality responses. For many applications, recognizing that a synthetic response is a high-quality for a given prompt is significantly easier than generating the response from scratch. For instance, if the dataset contains arithmetic or coding problems, it is relatively easy to programmatically verify a correct answer.

Weak Labeler. Key to our work is the notion of a *weak labeler*, a function, that given any set of prompts produces responses with average quality bounded away from zero.

To formally define it, we use an auxiliary function $a_P : P \rightarrow \mathcal{Y}$, which generates labels for all prompts in a set P .

Definition 3 (β -weak Labeler). Let $\beta \in [0, 1]$. The function labeler_β takes as input a prompt set $P \subset \mathcal{X}$, and uses an auxiliary function $a_P : \mathcal{X} \rightarrow \mathcal{Y}$ to label every prompt in P . Formally,

$$\text{labeler}_\beta(P) = \{(x, y) : x \in P, y = a_P(x)\} \subset \mathcal{X} \times \mathcal{Y}$$

We say that the labeler is β -weak if a β fraction of these labels are high-quality, i.e., for any input prompt set P ,

$$\frac{|\{(x, y) \in \text{labeler}_\beta(P) : q(x, y) = 1\}|}{|P|} \geq \beta.$$

In our setting, each iteration of data generation is allowed to make one call to the weak labeler. The role of the labeler is to create new responses to a set of prompts. We are not prescriptive about how the labeler is implemented, only that it provides some β fraction of high-quality responses. The labeler does not need to indicate *which* prompts have been correctly labeled, nor does it need to correctly label a representative portion of its input. For example, the labeler is allowed to only correctly label the “easiest” prompts that it receives as input. We think of these responses as being produced by an exogenous process, such as consulting with a human directly, having a human correct or critique LLM responses, or any other framework for generating responses that are not purely synthetic.

4.3 Objective

Given these capabilities — the ability to synthesize data, assess synthetic data quality, and weakly label new data — the **goal** of our algorithm is to construct datasets $\mathcal{D}_1, \dots, \mathcal{D}_T$ so that

$$\lim_{T \rightarrow \infty} \Pr_{x \sim P, y \sim g_T(x)}[q(x, y) = 1] = 1 \tag{1}$$

where $x \sim P$ denotes that x is chosen uniformly at random from P , and $y \sim g_T(x)$ denotes that y is chosen from distribution $g_T(x)$. In other words, as the number of algorithm iterations grows large, the final LLM output by the algorithm returns a correct response to almost every prompt in P . Note that this objective is similar to the objective of classical boosting. Rather than use weak learners to construct a good hypothesis, we ask whether *strong learners* and *weak data* can be used to construct a model that provides high-quality results on all prompts.

5 Algorithm

We present an algorithm for learning an LLM from a mixture of synthetically generated and weakly labeled data that uses the capabilities introduced in Section 4.

The aforementioned algorithm generates synthetic responses from the last generation of LLM. Synthetic data generation is given multiple opportunities to produce good responses, which are noisily recognized by filter_γ . Prompts that are consistently paired with low-quality responses are passed into labeler_β , which provides a minimal amount of signal. A mixture of good synthetically